Read One Model with FastAPI

Let's now add a *path operation* to read a single model to our **FastAPI** application.

Path Operation for One Hero

Let's add a new *path operation* to read one single hero.

We want to get the hero based on the `id`, so we will use a **path parameter** `hero_id`.

> **Info**
>
> If you need to refresh how *path parameters* work, including their data validation, check the FastAPI docs about Path Parameters.

Python 3.10+

```python
from fastapi import FastAPI, HTTPException
from sqlmodel import Field, Session, SQLModel, create_engine, select

# Code here omitted 👈

@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int):
    with Session(engine) as session:
        hero = session.get(Hero, hero_id)
        if not hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        return hero
```

👀 **Full file preview**

Python 3.10+

```python
from fastapi import FastAPI, HTTPException
from sqlmodel import Field, Session, SQLModel, create_engine, select


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)


class HeroCreate(HeroBase):
    pass


class HeroPublic(HeroBase):
    id: int


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate):
    with Session(engine) as session:
        db_hero = Hero.model_validate(hero)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes():
    with Session(engine) as session:
        heroes = session.exec(select(Hero)).all()
        return heroes


@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int):
    with Session(engine) as session:
        hero = session.get(Hero, hero_id)
        if not hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        return hero
```

For example, to get the hero with ID `2` we would send a `GET` request to:

```
/heroes/2
```

Handling Errors

Then, because FastAPI already takes care of making sure that the `hero_id` is an actual integer, we can use it directly with `Hero.get()` to try and get one hero by that ID.

But if the integer is not the ID of any hero in the database, it will not find anything, and the variable `hero` will be `None`.

So, we check it in an `if` block, if it's `None`, we raise an `HTTPException` with a `404` status code.

And to use it, we first import `HTTPException` from `fastapi`.

This will let the client know that they probably made a mistake on their side and requested a hero that doesn't exist in the database.

Python 3.10+

```python
from fastapi import FastAPI, HTTPException
from sqlmodel import Field, Session, SQLModel, create_engine, select

# Code here omitted 👈

@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int):
    with Session(engine) as session:
        hero = session.get(Hero, hero_id)
        if not hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        return hero
```

👀 **Full file preview**

Python 3.10+

```python
from fastapi import FastAPI, HTTPException
from sqlmodel import Field, Session, SQLModel, create_engine, select


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)


class HeroCreate(HeroBase):
    pass


class HeroPublic(HeroBase):
    id: int


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate):
    with Session(engine) as session:
        db_hero = Hero.model_validate(hero)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes():
    with Session(engine) as session:
        heroes = session.exec(select(Hero)).all()
        return heroes


@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int):
    with Session(engine) as session:
        hero = session.get(Hero, hero_id)
        if not hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        return hero
```

Return the Hero

Then, if the hero exists, we return it.

And because we are using the `response_model` with `HeroPublic`, it will be validated, documented, etc.

Python 3.10+

```python
from fastapi import FastAPI, HTTPException
from sqlmodel import Field, Session, SQLModel, create_engine, select

# Code here omitted 👈

@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int):
    with Session(engine) as session:
        hero = session.get(Hero, hero_id)
        if not hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        return hero
```

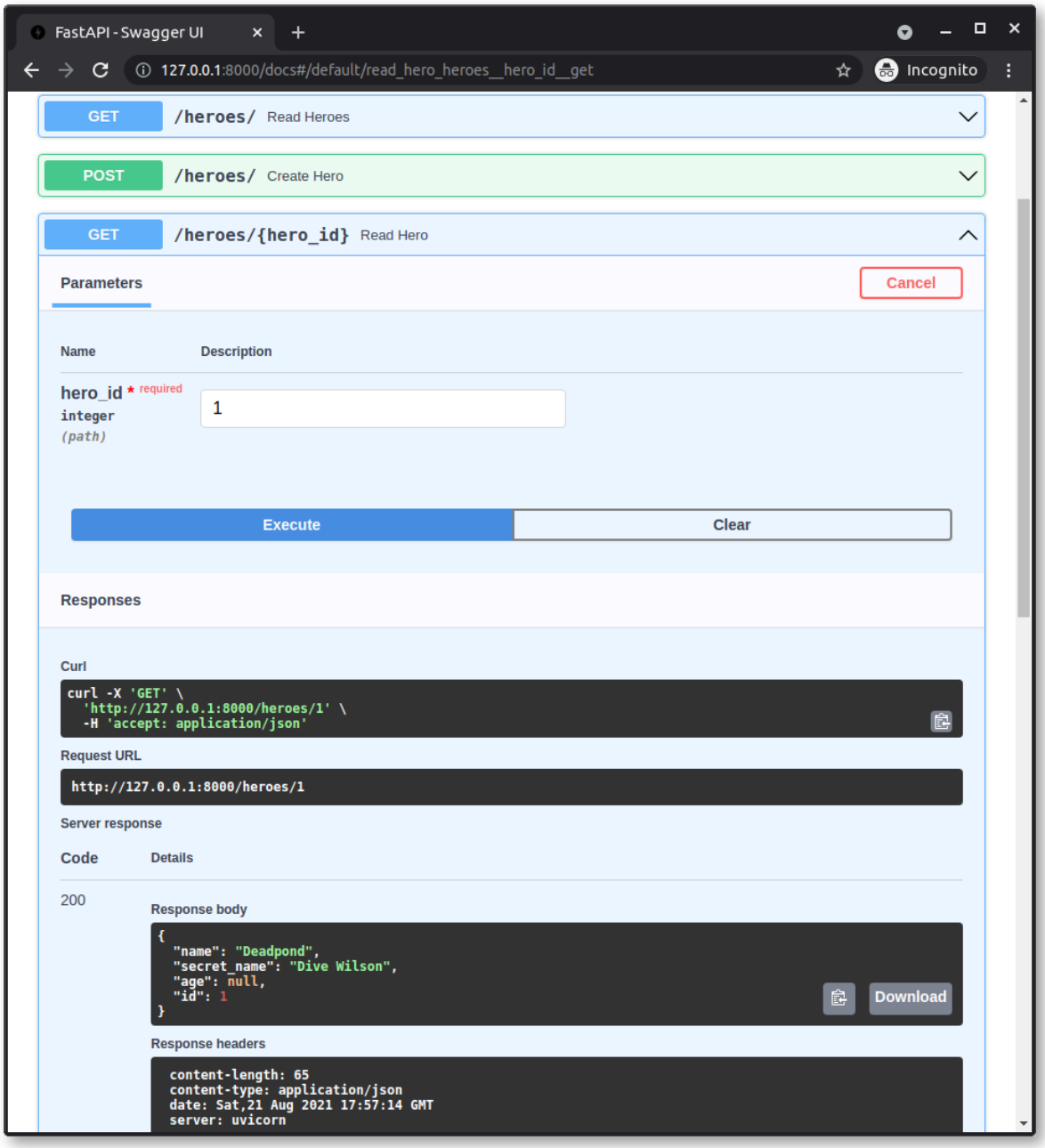
👀 **Full file preview**

Python 3.10+

```python
from fastapi import FastAPI, HTTPException
from sqlmodel import Field, Session, SQLModel, create_engine, select


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)


class HeroCreate(HeroBase):
    pass


class HeroPublic(HeroBase):
    id: int


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate):
    with Session(engine) as session:
        db_hero = Hero.model_validate(hero)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes():
    with Session(engine) as session:
        heroes = session.exec(select(Hero)).all()
        return heroes


@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int):
    with Session(engine) as session:
        hero = session.get(Hero, hero_id)
        if not hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        return hero
```

Check the Docs UI

We can then go to the docs UI and see the new *path operation*.



Recap

You can combine **FastAPI** features like automatic path parameter validation to get models by ID.